DocSense: LLM-Powered Document Assistant Using Retrieval-Augmented Generation (RAG)

Project Report

Group 14

Team Members:
Sai Varun Chetrypally
Sriyashveer Reddy Pulapalli
Rishitha Challa

Faculty Coach:
Dr. Feng George Yu

INTRODUCTION

DocSense is an AI-powered document question-answering system that allows users to interact with unstructured documents through a simple chat-based interface. With the increasing amount of digital content such as research papers, reports, and technical documents, it has become difficult for users to quickly find relevant information. Traditional keyword-based search methods often return incomplete or less relevant results because they do not fully understand the context of the query.

To overcome this limitation, DocSense uses Retrieval-Augmented Generation (RAG), which combines information retrieval with large language models (LLMs). Instead of relying only on pre-trained knowledge, the system retrieves relevant content directly from the uploaded document and uses it to generate accurate and context-aware answers. This approach ensures that the responses are grounded in the actual document, improving both reliability and transparency.

In this system, users can upload PDF documents and ask questions in natural language. The backend processes the document by extracting text and tables, splitting the content into smaller chunks, generating embeddings, and performing semantic search to find relevant information. The frontend provides a simple and interactive chat interface for users. The system also displays source references such as file name, page number, and chunk ID, which helps users verify the answers.

Overall, DocSense demonstrates how combining retrieval techniques with generative AI can create a practical and reliable document assistant for understanding complex information.

BACKGROUND

The rapid advancement of Artificial Intelligence, particularly Large Language Models (LLMs), has significantly improved the ability of machines to understand and generate human-like text. Models such as GPT have demonstrated strong performance in tasks like question answering, summarization, and conversational systems. However, despite their capabilities, LLMs have certain limitations. They rely on pre-trained knowledge, which may be outdated, and they can sometimes generate incorrect or misleading information, a phenomenon commonly referred to as hallucination. Additionally, LLMs do not have direct access to external or user-specific documents unless explicitly provided.

To address these challenges, Retrieval-Augmented Generation (RAG) has emerged as an effective approach. RAG combines information retrieval techniques with generative models to improve the accuracy and relevance of responses. Instead of generating answers purely from internal knowledge, the system first retrieves relevant information from a document or database and then uses that information as context for the language model to generate a response. This ensures that the answers are grounded in real data and can be traced back to their source.

In the context of document question answering, RAG is particularly useful because it allows systems to work with large and unstructured data sources such as PDFs. By converting document content into vector embeddings and storing them in a vector database, semantic similarity search can be performed to identify the most relevant sections of the document. These retrieved sections are then used by the language model to produce accurate and context-aware answers.

Overall, the combination of LLMs and retrieval mechanisms forms the foundation of modern document-based AI systems, enabling more reliable, explainable, and scalable solutions for information access.

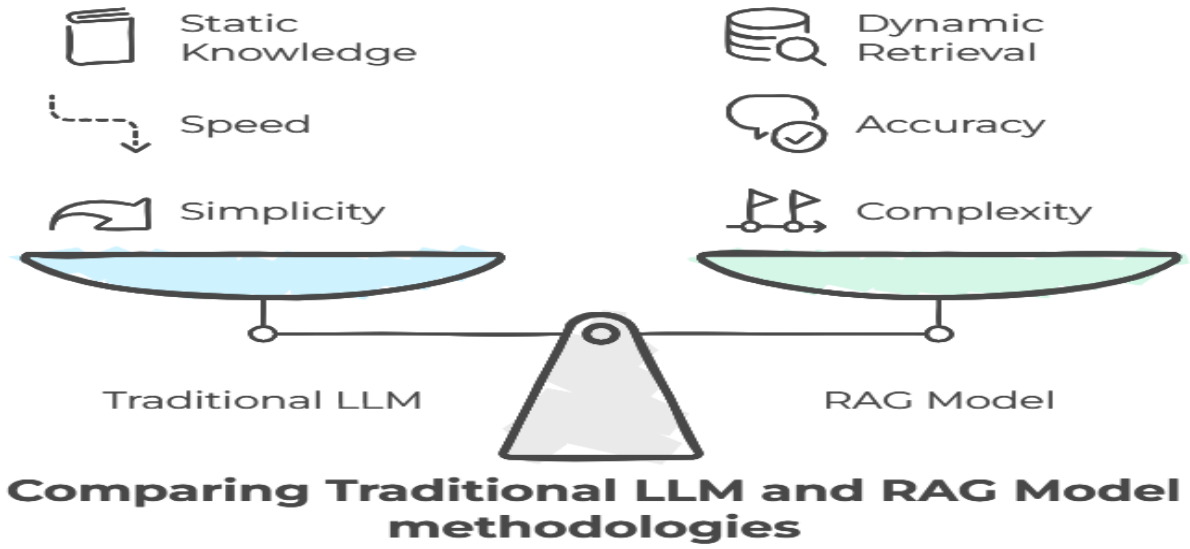

Figure 1: Comparing LLM and RAG Model Methodologies

PROBLEM STATEMENT

With the increasing availability of digital documents such as research papers, reports, and technical manuals, users often face difficulty in quickly extracting relevant information. Traditional search methods, which rely on keyword matching, are not effective in understanding the context or intent behind a user's query. As a result, users must manually read through large volumes of text to find the required information, which is time-consuming and inefficient.

Although Large Language Models (LLMs) provide advanced capabilities for natural language understanding and generation, they are not designed to directly access or process user-specific documents. They may also produce inaccurate or hallucinated responses when answering knowledge-intensive questions. This lack of reliability and inability to reference source information limits their effectiveness in real-world document analysis tasks.

Therefore, there is a need for a system that can accurately retrieve relevant information from user-provided documents and generate context-aware answers while maintaining transparency. The goal of this project is to develop an AI-powered document assistant that integrates retrieval mechanisms with language models to enable efficient, reliable, and explainable document question answering.

METHODOLOGY

To address the problem of extracting meaningful information from unstructured PDF documents, we designed and implemented DocSense, a Retrieval-Augmented Generation (RAG)-based system. The methodology follows a structured pipeline consisting of three major phases: Ingestion, Retrieval, and Synthesis, as illustrated in Figure 2.

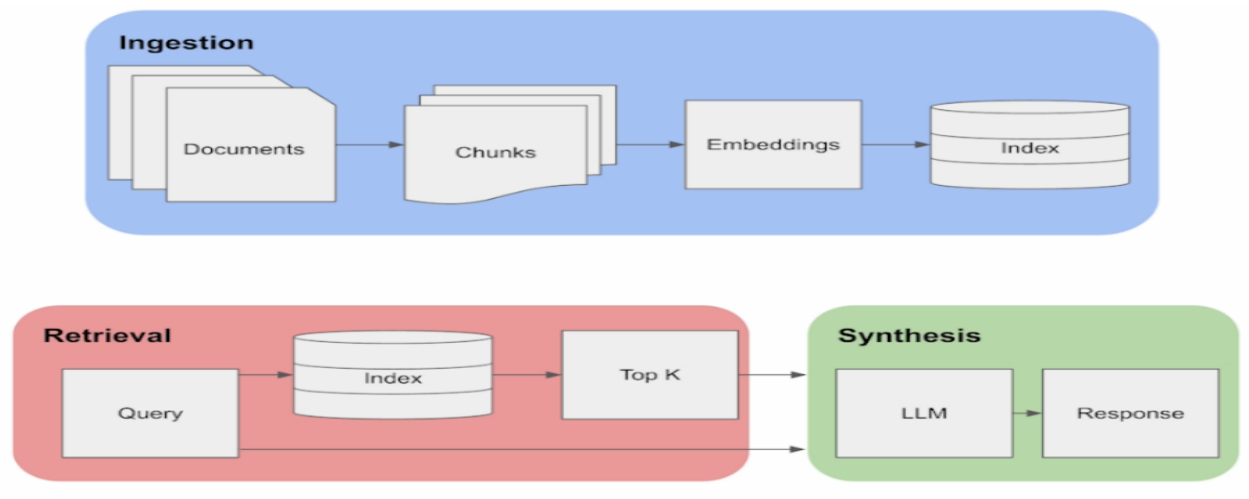

Figure 2: Retrieval-Augmented Generation (RAG) Architecture used in DocSense

1. INGESTION PHASE

The ingestion phase is responsible for converting raw PDF documents into a structured and searchable format. When a user uploads a PDF through the interface, the backend system processes the document by extracting textual and tabular content using a PDF parsing library.

The extracted content is then divided into smaller segments known as chunks. Chunking is performed to ensure that the information remains within the token limits of embedding models and improves retrieval accuracy. In our implementation, we used fixed-size chunks with slight overlap to preserve context continuity.

Each chunk is then transformed into a vector embedding using a pre-trained sentence transformer model (*all-MiniLM-L6-v2*). These embeddings capture the semantic meaning of the text rather than just keyword-based representations.

Finally, all embeddings, along with metadata such as source file name, page number, and chunk ID, are stored in a ChromaDB vector database. This completes the indexing process and prepares the data for efficient retrieval.

2. RETRIEVAL PHASE

The retrieval phase begins when the user submits a question through the chat interface. The input query is first converted into a vector embedding using the same embedding model to ensure consistency with the stored document embeddings.

The system then performs a similarity search in the vector database to identify the most relevant chunks. This is done using distance-based metrics such as cosine similarity.

From the search results, the system selects the top-k most relevant chunks, which serve as contextual information for answering the user's query. This step ensures that only the most meaningful and relevant portions of the document are passed to the language model, reducing noise and improving answer quality.

3. SYNTHESIS PHASE

In the synthesis phase, the retrieved chunks and the user's question are combined and passed to a large language model (LLM) via the OpenAI API.

The LLM generates a context-aware response by grounding its answer strictly in the retrieved document content. This ensures that the output is not based on general knowledge alone but is directly supported by the uploaded PDF.

Additionally, the system provides source attribution, including file name, page number, and chunk reference, allowing users to verify the origin of the generated response. This improves transparency and trust in the system.

4. SYSTEM WORKFLOW SUMMARY

Overall, the system follows a sequential pipeline:

1. User uploads a PDF document
2. Document is processed and converted into chunks
3. Chunks are transformed into embeddings and stored in a vector database
4. User submits a query
5. Relevant chunks are retrieved using similarity search
6. Retrieved context is passed to the language model
7. A grounded and explainable response is generated

5. DESIGN CONSIDERATIONS

Several design decisions were made to improve system performance:

- **Temporary Vector Storage:** Each uploaded PDF replaces the previous dataset to ensure session-specific querying

- **Chunking Strategy:** Overlapping chunks were used to maintain context
- **Top-K Retrieval:** Limits unnecessary data sent to the LLM
- **Source Transparency:** Improves explainability of responses

EXPERIMENTS

The goal of the experiments is to evaluate the functionality, accuracy, and reliability of the DocSense system in answering questions based on uploaded PDF documents. The system was tested using multiple research papers to validate its ability to process documents, retrieve relevant information, and generate context-aware responses.

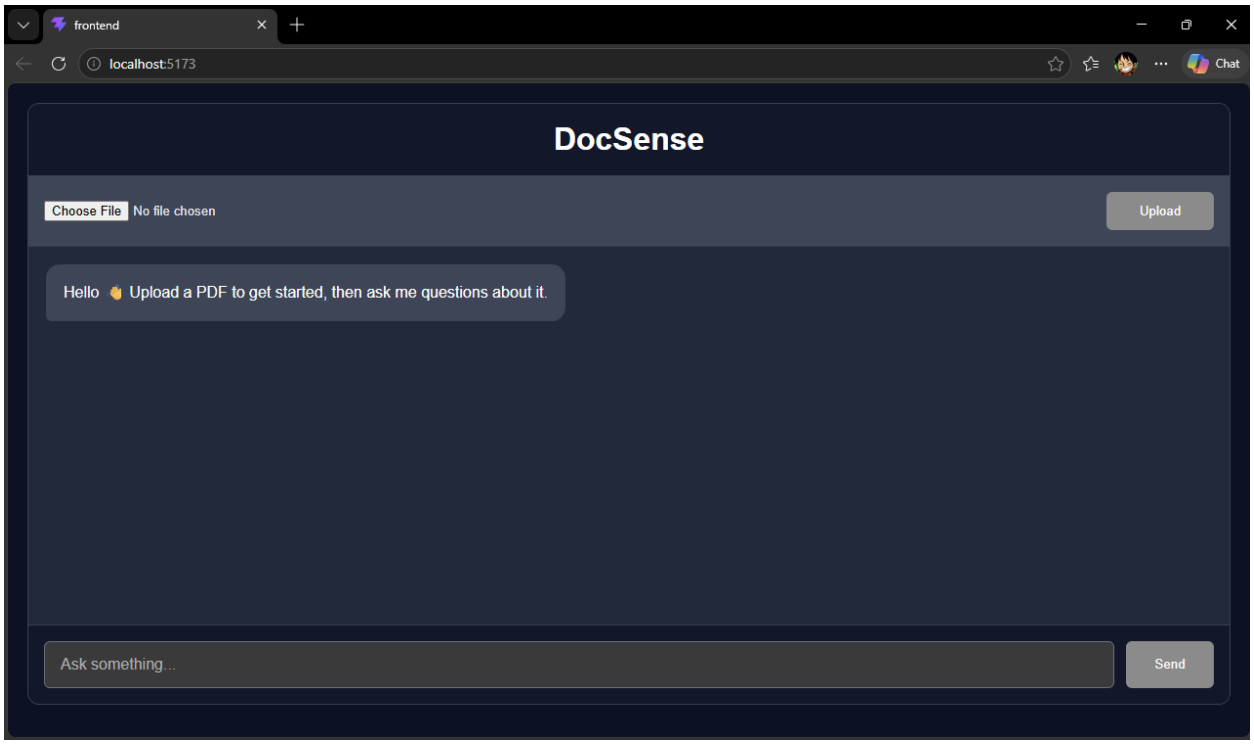

Figure 3: Main Page

1. EXPERIMENTAL SETUP

The system was implemented using the following setup:

- **Backend:** FastAPI (Python)
- **Frontend:** React (Vite)
- **Embedding Model:** all-MiniLM-L6-v2 (Sentence Transformers)
- **Vector Database:** ChromaDB
- **Language Model:** OpenAI API

- **Development Environment:** Local system (Windows)

Two different PDF documents were used for testing:

1. *2312.10997v5.pdf* (RAG-related research paper)
2. *Artificial Intelligence in Information Systems State of the Art.pdf*

2. TEST CASE 1: RAG-BASED DOCUMENT

Input

- Uploaded PDF: *2312.10997v5.pdf*
- Question: *"What are the three paradigms of RAG?"*

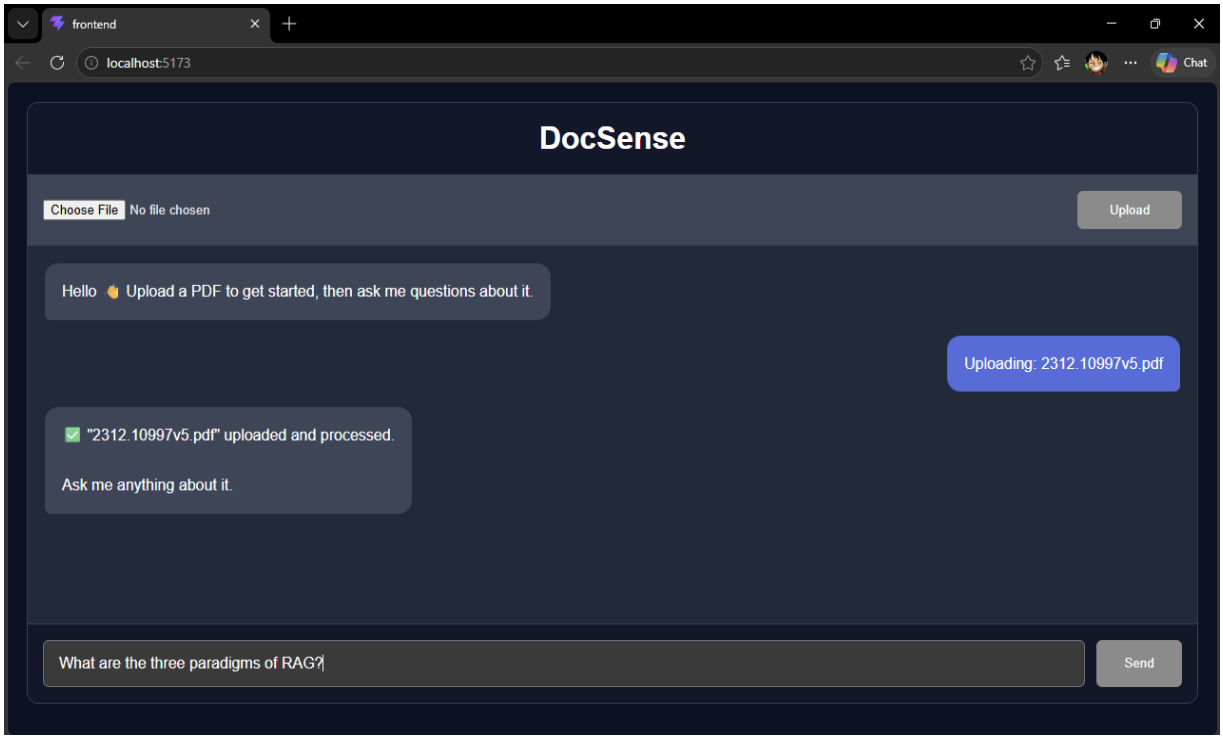

Figure 4: PDF Upload Successful

Output

The system correctly identified and generated the answer:

- Naive RAG
- Advanced RAG
- Modular RAG

It also provided source references including page numbers and chunk IDs.

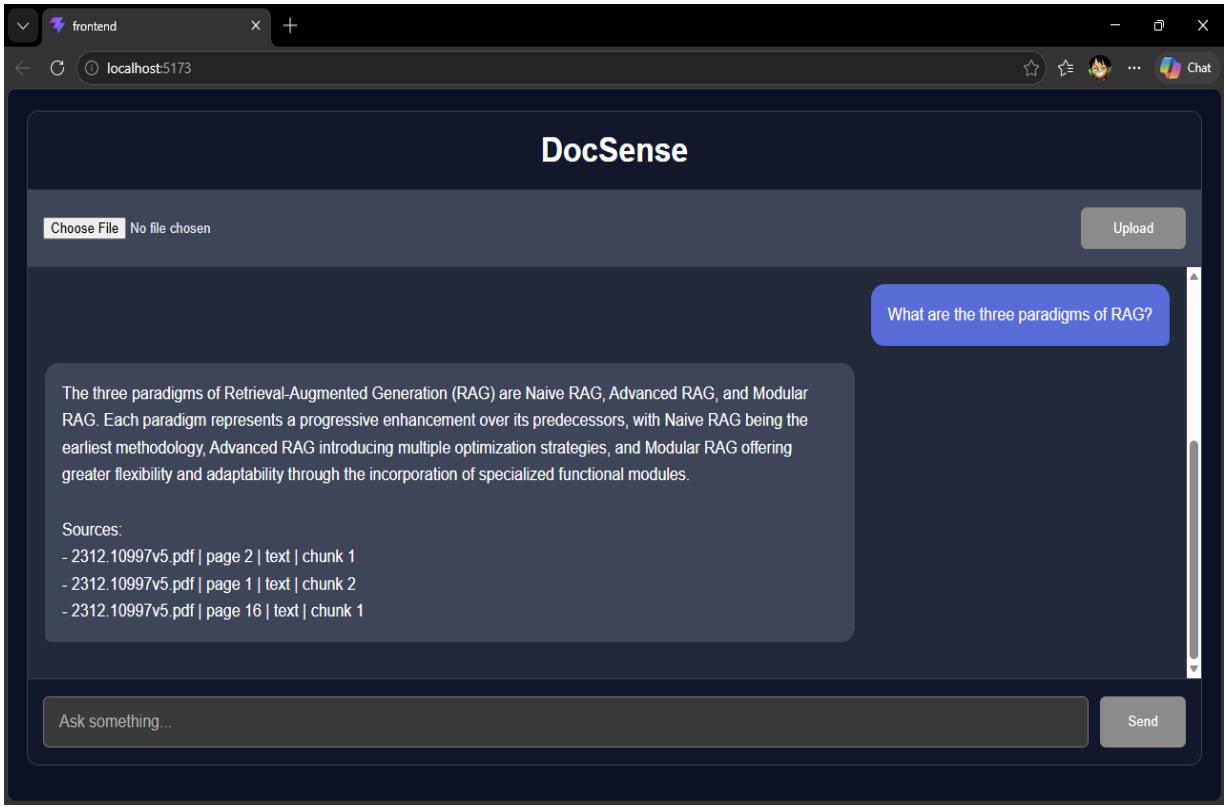

Figure 5: Response to the Question from OpenAI

3. TEST CASE 2: CONCEPTUAL COMPARISON

Input

- Question: *"What is the difference between Naive RAG and Modular RAG?"*

Output

The system generated a detailed comparison:

- Naive RAG: simple pipeline (index → retrieve → generate)
- Modular RAG: flexible, supports multiple modules and adaptive retrieval

The response was:

- Context-aware
- Structured
- Based only on documents

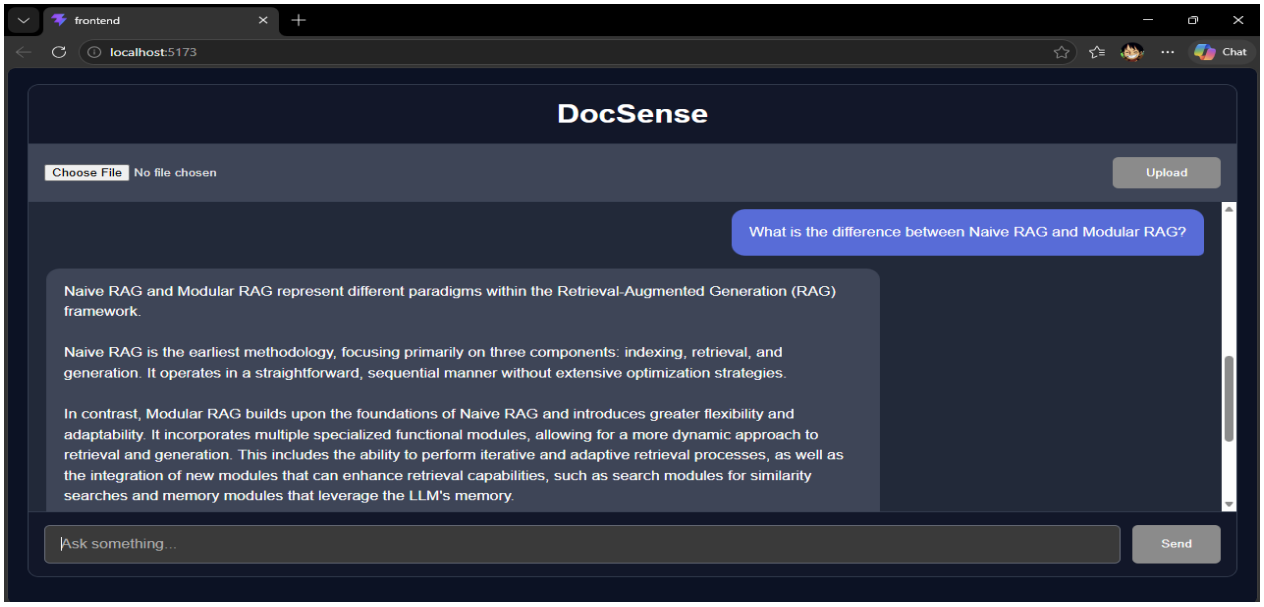

Figure 6: Response to the Question from OpenAI

4. TEST CASE 3: DIFFERENT DOCUMENT CONTEXT

Input

- Uploaded PDF: *Artificial Intelligence in Information Systems State of the Art.pdf*
- Question: *"What criteria are used for judging good AI research?"*

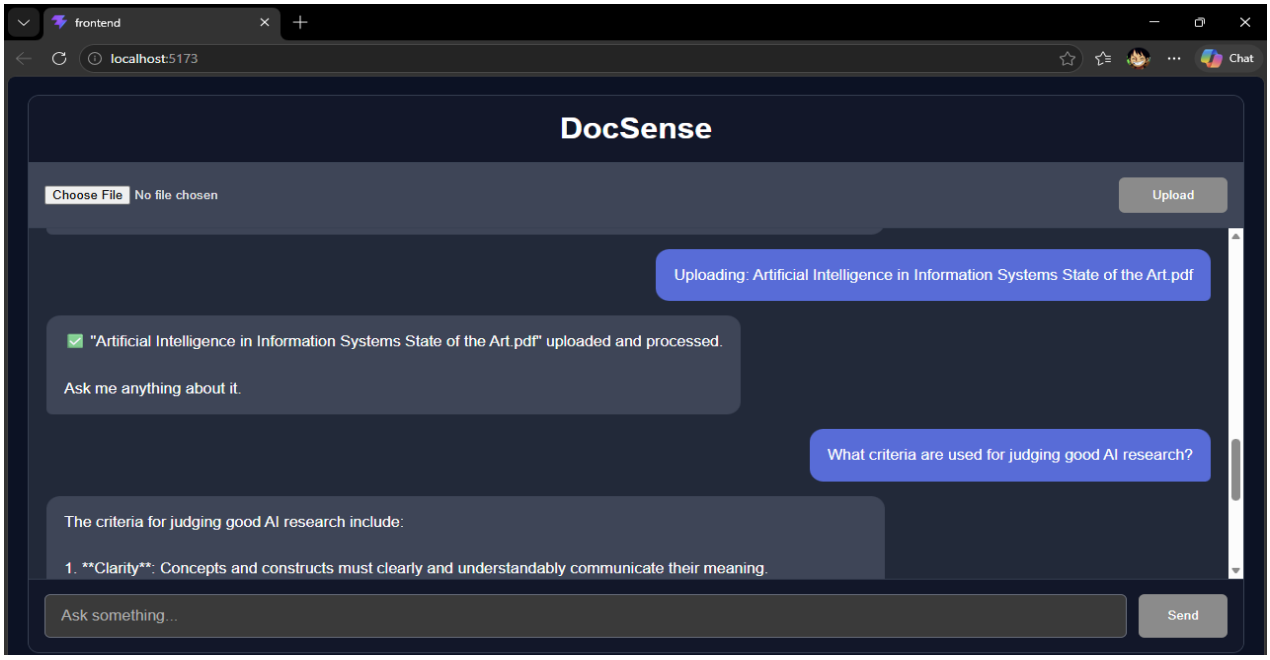

Figure 7: Response to the Question from OpenAI

Output

The system successfully extracted:

- Clarity
- Theoretical Glue
- Cumulative Tradition
- Parsimony
- Applicability

This confirms that the system adapts to different documents dynamically.

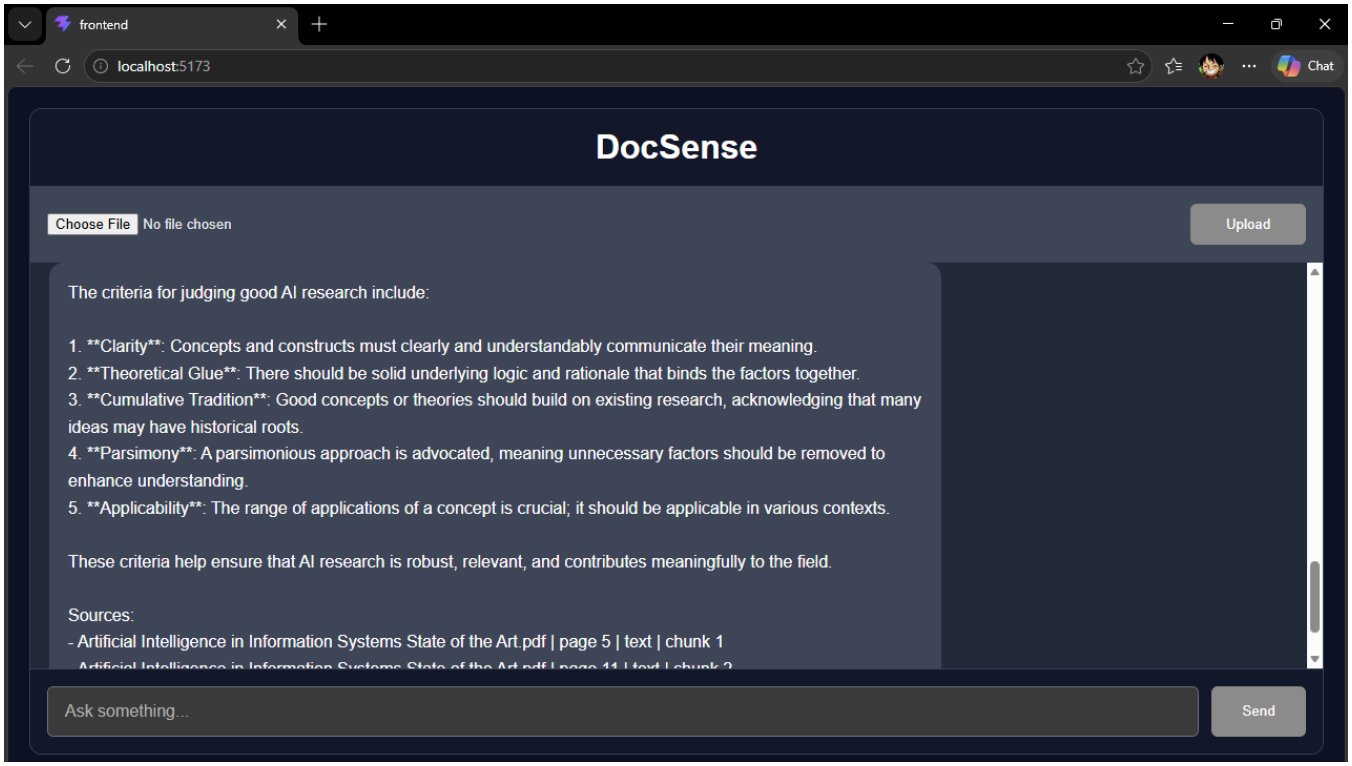

Figure 8: Response to the Question from OpenAI

5. TEST CASE 4: OUT-OF-SCOPE QUESTION

Input

- Question: *"What is the weather today?"*

Output

The system responded:

"I do not have enough information in the provided documents to answer that."

This demonstrates that:

- The system avoids hallucination
- Only uses document-based knowledge

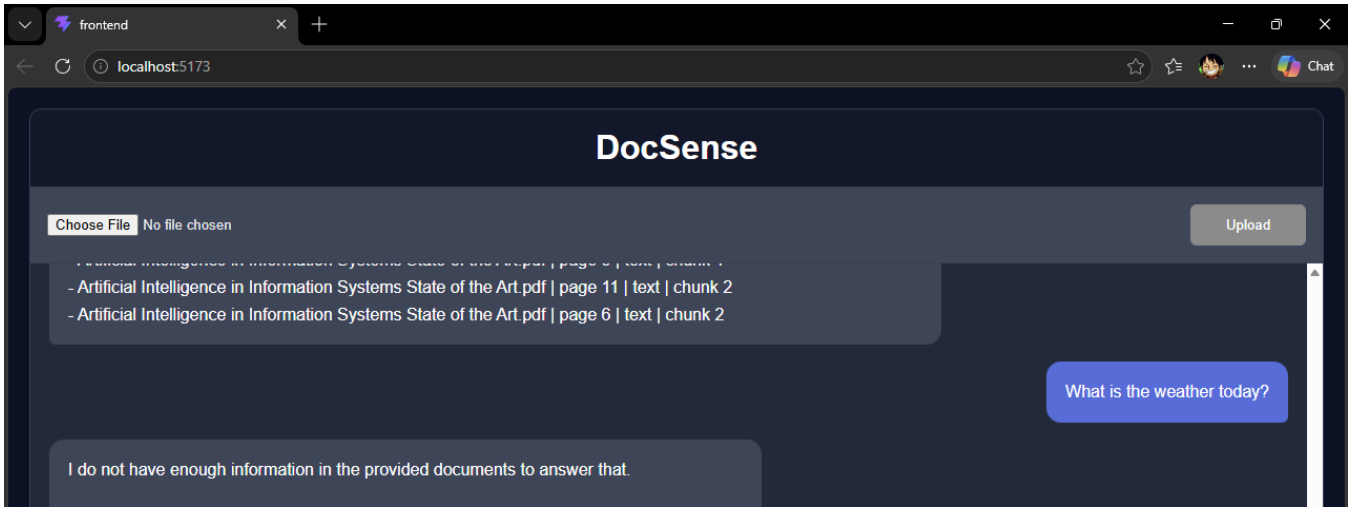

Figure 9: Response to the Question from OpenAI

6. PERFORMANCE OBSERVATIONS

- **Processing Time:**
  - PDF upload + embedding: few seconds depending on size
- **Query Response Time:**
  - Typically, 1–3 seconds
- **Accuracy:**
  - High when relevant content exists in document
- **Failure Handling:**
  - Gracefully handles irrelevant queries

7. EVALUATION SUMMARY

The system was evaluated based on:

- **Retrieval Quality:** Relevant chunks were consistently selected
- **Answer Relevance:** Responses directly addressed the question
- **Faithfulness:** Answers were grounded in document content
- **Explainability:** Source citations improved transparency

8. KEY FINDINGS

- The RAG approach significantly improves answer accuracy compared to standalone LLMs
- Chunking and embedding play a critical role in retrieval performance
- Limiting responses to document context prevents misinformation
- The system works effectively across different types of documents

DISCUSSION

The DocSense system demonstrates how Retrieval-Augmented Generation (RAG) can effectively bridge the gap between static document storage and interactive question-answering systems. By combining semantic retrieval with language model generation, the system can produce responses that are both contextually relevant and grounded in the uploaded document.

One of the key strengths observed during experimentation is the system's ability to retrieve meaningful context from large documents and generate accurate answers. Unlike traditional keyword-based search systems, DocSense uses vector embeddings to capture the semantic meaning of text, allowing it to identify relevant information even when the wording of the question differs from the document.

Another important observation is the system's controlled response behavior. When asked questions that are not related to the uploaded document, the system correctly responds that it does not have sufficient information. This highlights an important advantage of the RAG approach it reduces hallucination and ensures that responses remain faithful to the provided data.

The system also demonstrates flexibility across different documents. During testing, DocSense was able to process multiple research papers and answer domain-specific questions without requiring any retraining. This shows that the approach is scalable and adaptable to different types of textual data.

However, some limitations were identified. The performance of the system depends heavily on the quality of chunking and embeddings. If important context is split incorrectly during chunking, it may affect retrieval accuracy. Additionally, the system currently processes one document at a time, meaning it does not support multi-document querying or long-term session memory.

Another limitation is related to processing time and resource usage. While the system performs efficiently for small to medium-sized documents, larger PDFs may increase embedding time and response latency. This suggests that further optimization, such as caching or incremental indexing, could improve performance.

Overall, the results indicate that DocSense successfully implements a practical RAG-based pipeline and achieves its objective of enabling document-based question answering. The system provides accurate, explainable, and context-aware responses, making it a strong foundation for future enhancements such as multi-document support, improved UI, and advanced retrieval techniques.

CONCLUSION

In this project, we developed **DocSense**, an AI-powered document question-answering system using the Retrieval-Augmented Generation (RAG) approach. The system was designed to address the challenge of extracting relevant information from unstructured PDF documents in an efficient and reliable manner. By integrating document retrieval with large language models, DocSense enables users to interact with documents through a simple chat-based interface.

The implementation demonstrates a complete end-to-end pipeline, including document ingestion, text and table extraction, chunking, embedding generation, vector storage, semantic retrieval, and response generation. The use of vector embeddings and similarity search allows the system to understand the context of user queries, while the integration with a language model ensures that the generated responses are coherent and meaningful.

Experimental results show that the system can generate accurate, context-aware answers that are grounded in the uploaded document. The inclusion of source references further enhances transparency and allows users to verify the information. Additionally, the system effectively avoids answering out-of-scope questions, reducing the risk of hallucination.

Overall, DocSense successfully demonstrates the effectiveness of the RAG framework for document-based question answering. The project highlights the potential of combining retrieval mechanisms with generative AI to build practical and explainable intelligent systems. Future improvements could include support for multiple documents, enhanced user interface design, and optimization of retrieval and processing performance.

TEAM CONTRIBUTIONS

The project was developed collaboratively, with each member contributing to different components of the system:

- **Sai Varun Chetrypally** – Worked on the core system pipeline, including document parsing, text and table extraction, chunking, embedding generation, and integration of the retrieval and response generation workflow. Also contributed to system integration, debugging, and overall functionality of the end-to-end pipeline.
- **Sriyashveer Reddy Pulapalli** – Contributed to backend development and API implementation using FastAPI, including handling user requests and supporting system integration.
- **Rishitha Challa** – Developed the vector database and retrieval system, including embedding storage using ChromaDB and implementation of semantic similarity search.

REFERENCES

1. P. Lewis, E. Perez, A. Piktus, et al., *"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,"* 2020.
2. Y. Gao, Y. Xiong, X. Gao, et al., *"Retrieval-Augmented Generation for Large Language Models: A Survey,"* 2023.
3. Z. Li, *"Retrieval-Augmented Generation for Educational Applications: A Survey,"* 2025.
4. S. Es, J. James, L. Espinosa-Anke, et al., *"RAGAS: Automated Evaluation of Retrieval-Augmented Generation,"* 2023.
5. A. Brown, M. Roman, B. Devereux, *"A Systematic Literature Review of Retrieval-Augmented Generation: Techniques, Metrics, and Challenges,"* 2025.
6. D. Wampler, D. Nielson, A. Seddighi, *"Engineering the RAG Stack: Architecture and Trust Frameworks,"* 2025.
7. *"Retrieval-Augmented Generation,"* general concept overview

APPENDIX

The complete implementation of the DocSense system, including source code, documentation, and project components, is available in the GitHub repository: https://github.com/Varunch-16/docsense. The repository contains the backend developed using FastAPI, the frontend built with React, and all modules related to document ingestion, chunking, embedding generation, vector storage, retrieval, and response generation. To evaluate the system, several test queries were used across different documents. For the RAG research paper, example queries included "What are the three paradigms of RAG?" and "What is the difference between Naive RAG and Modular RAG?", while for the AI research paper, queries such as "What criteria are used for judging good AI research?" were tested. Additionally, out-of-scope queries like "What is the weather today?" were used to verify that the system correctly avoids generating unsupported answers. The system produces responses in a structured format that includes a generated answer followed by clearly cited sources, specifying the file name, page number, content type (text or table), and chunk ID to ensure transparency and traceability. The backend exposes key API endpoints, including /upload for uploading and processing PDF documents, /ask for querying the processed content, and /health for checking system status. The system is designed to handle one document at a time, where each new upload replaces the previously processed data to maintain relevance. For security and proper usage, users are required to provide their own OpenAI API key during runtime, and no sensitive credentials are stored within the project repository.